Design and Implementation of an Automated Wind-Tunnel Platform for Heatsink Thermal Characterisation

A Bachelor's Thesis in Engineering

Author: Alexander Groenvynck **Programme:** Bachelor in Energy Technology (Odisee University of Applied Sciences, Ghent) **Host institution:** University of Girona (exchange) **Supervisor:** Lino Montoro Moreno **Academic year:** 2025-2026

Abstract

Heatsinks are the dominant passive thermal-management solution in electronics, yet their comparative evaluation in a teaching laboratory is typically slow, manual, and error-prone. This thesis presents the design, implementation, and validation of an automated wind-tunnel platform that characterises the thermal performance of heatsinks under controlled forced-convection conditions. The system couples a ceramic heater and a controllable fan to an ESP32-S3 microcontroller running a PID control loop, with a browser-based graphical interface that reduces the entire test procedure to a single button press. The platform steps through a set of target temperatures, detects thermal equilibrium automatically, records steady-state power and temperature, and exports the results as a CSV file - all without operator intervention between set-points.

The contribution of this work is primarily a software-and-systems contribution: the firmware exposes a stable serial interface and is treated as a locked black box, while a FastAPI backend and single-file frontend deliver a zero-friction, safety-aware student workflow. On validation the system successfully drove a heatsink to each programmed set-point, detected equilibrium reliably within the ± 0.5 °C stability criterion, and produced a complete result CSV without manual steps. Post-test heater shutdown fired on every termination path tested.

Keywords: heatsink, thermal characterisation, forced convection, wind tunnel, PID control, ESP32, embedded systems, instrumentation, laboratory automation.

Acknowledgements

I would like to thank my supervisor, **Lino Montoro Moreno**, for his guidance throughout this project and in particular for his hands-on help in the design and construction of the wind-tunnel enclosure.

I also want to acknowledge two fellow students who contributed to the project. **Miquel Lladó i Tuñà** was involved from the early stages and helped build the heater module - specifically the copper spreader plate, the steel backing plate, and the insulation material beneath the heater element. **Esmeralda Solís** is continuing related work on the project by simulating the thermal behaviour and heat-flow properties of the heatsinks in Ansys Workbench, which will complement the experimental measurements this platform is designed to produce.

Declaration on the Use of AI Tools

The software components of this project - the ESP32 firmware, the Python/FastAPI backend, and the browser-based frontend - were developed with the assistance of an AI coding assistant (Claude Code), used under my direction and supervision. I specified the requirements, made the design decisions, integrated and tested the code, and am responsible for the correctness of the final system.

The physical platform - the wind tunnel, heater module, sensor mounting, electrical wiring, and assembly - was designed, built, and wired by me. All hardware validation and testing described in this thesis was carried out by me on the physical apparatus.

This text was written by me; AI assistance used in drafting was reviewed and verified for accuracy.

Table of Contents

List of Figures

List of Tables

Nomenclature and Abbreviations

Symbol	Meaning	Unit
T	Heatsink base temperature	°C
T_amb	Ambient temperature	°C
ΔT	Temperature rise above ambient, T - T_amb	°C
P	Electrical power dissipated in the heater	W
R_th	Thermal resistance, ΔT / P	°C/W
PWM	Pulse-width-modulated heater drive (0-255)	-
EqPWM	Equilibrium PWM estimate (steady-state)	-
PID	Proportional-Integral-Derivative controller	-
EMA	Exponential moving average filter	-

Abbreviation	Expansion
MCU	Microcontroller unit
SPI	Serial Peripheral Interface
I²C	Inter-Integrated Circuit bus
GUI	Graphical user interface
CSV	Comma-separated values
NVS	Non-volatile storage (ESP32)

1. Introduction

1.1 Background and motivation

Thermal management is a constraining factor in nearly all modern electronics. As power densities rise, the ability of a heatsink to move heat away from a component directly determines reliability and performance. In an engineering teaching laboratory, students are expected to develop an intuition for why one heatsink outperforms another - fin geometry, material, surface area, and airflow regime all matter - and ideally to quantify those differences experimentally.

In practice, measuring heatsink performance by hand is awkward. It requires holding a heat source at a stable temperature, waiting for thermal equilibrium, reading several instruments simultaneously, and recording results without transcription errors - all while a live heating element is energised. The cognitive and procedural overhead crowds out the actual learning objective: comparing heatsinks.

1.2 Technical background

Two concepts underpin the whole project.

Thermal resistance. The standard figure of merit for a heatsink is its thermal resistance, defined as the temperature rise above ambient per unit of dissipated power:

$R_{th} = \Delta T / P$ (°C/W), where $\Delta T = T - T_{amb}$.

A lower R_{th} means heat is carried away more effectively, i.e. a better heatsink. For the comparison to be fair, the airflow over the heatsink must be the same for every sample - which is the reason for a controlled wind tunnel rather than still air.

Closed-loop temperature control. To hold the heater at a chosen temperature the system uses a PID (Proportional-Integral-Derivative) controller, a standard feedback method that continuously adjusts the heater power based on the difference between the measured and target temperature. The student never needs to understand or tune the PID; it is configured once and hidden behind the automated workflow.

1.3 Project context

This thesis documents the **HeatsinkLab Wind Tunnel**, a platform designed so that a student with no embedded-systems or control-theory background can characterise a heatsink in minutes. A ceramic heater drives an instrumented copper block to a series of target temperatures while a fan provides controlled forced convection through a small wind tunnel. An ESP32-S3 microcontroller runs the closed-loop control and streams telemetry over USB to a browser-based interface, which automates the test and exports the data.

The guiding design principle is **software-first development**: the firmware is treated as a stable, locked black box exposing a serial command and telemetry interface, while all user-facing features and workflow logic live in the backend and frontend. This keeps iteration fast - a software change deploys by restarting a script, not by re-flashing hardware - and keeps the safety-critical control loop unchanged once validated.

1.4 Aim and scope

The aim of this work is to deliver a complete, safe, and repeatable system that lets students obtain comparative thermal data for heatsinks through a single-button workflow. The scope covers:

- the system architecture and the rationale for the three-tier split;
- the control strategy used to reach and detect thermal equilibrium;
- the instrumentation and data pipeline from sensor to exported CSV;
- the automated test workflow and its safety behaviour;
- functional validation of the complete platform.

Out of scope are an independent hardware safety watchdog, closed-loop airspeed control, and the full suite of environmental sensors that the architecture already accommodates but which were not wired during this project.

1.5 Thesis outline

Chapter 2 states the problem and research questions. Chapter 3 details the methodology and system design. Chapter 4 presents the validation. Chapter 5 discusses the findings, Chapter 6 concludes, and Chapter 7 sets out recommendations and future work.

---

2. Problem Statement and Research Questions

2.1 Problem statement

A teaching laboratory needs to compare the thermal performance of different heatsinks *fairly* (same conditions), *repeatably* (same result on re-test), and *safely* (a live heater is involved), while keeping the procedure simple enough that students focus on the engineering result rather than on operating the apparatus. No off-the-shelf, low-cost instrument delivers this combination, and a fully manual rig fails on simplicity, repeatability, and safety simultaneously.

2.2 Research questions

- **RQ1.** Can a low-cost microcontroller platform hold a heat source at a series of set-point temperatures with sufficient stability for a repeatable measurement?
- **RQ2.** Can thermal equilibrium be detected *automatically* and reliably, removing the need for an operator to judge when the system has settled?
- **RQ3.** Can the complete test workflow be reduced to a single student action while guaranteeing safe shutdown of the heater under all termination conditions?
- **RQ4.** Does the platform produce a comparative thermal metric that meaningfully distinguishes heatsinks of differing geometry and material?

2.3 Requirements

ID	Requirement	Type
R1	Reach and hold set-point temperatures in the lab range (40-80 °C)	Functional
R2	Detect thermal equilibrium automatically (±0.5 °C / 8 s stability window)	Functional

ID	Requirement	Type
R3	Record steady-state temperature and electrical power at each set-point	Functional
R4	Export results as a CSV with a stable, versioned schema	Functional
R5	Heater must be de-energised automatically on test completion, manual stop, or fault	Safety
R6	Provide a single-button student workflow	Usability
R7	Operate without hardware via a virtual simulator	Non-functional
R8	Be extensible to airflow, pressure, and humidity sensors	Non-functional

3. Methodology and System Design

3.1 System architecture

The platform is organised into three tiers with a strict interface between each (Figure 3.1). The safety-critical control loop is isolated in firmware and validated once; all student-facing features are developed in software that can be changed and redeployed without touching the hardware.

```
Firmware (C++ / ESP32-S3)          src/  - stable, black box
· PID heater + fan control
· MAX6675 thermocouple (SPI)
· INA226 power monitor (I²C)
· Serial command interface

            USB Serial @ 115 200 baud

Backend (Python / FastAPI)         tools/web_gui/server.py
· WebSocket server :8765
· Telemetry parser + CSV logger
· Virtual MCU simulator
```

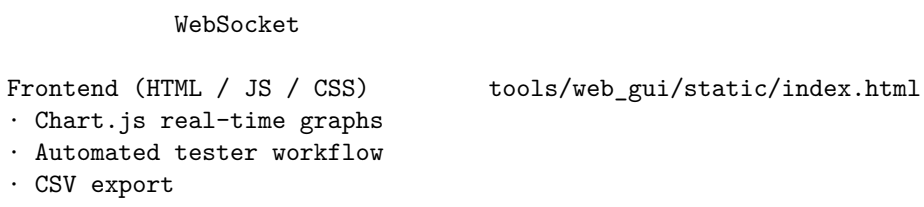

Figure 3.1 - Three-tier architecture. The only contracts between tiers are the serial command/telemetry protocol (firmware backend) and the WebSocket message set (backend frontend).

3.2 Physical construction and instrumentation

The mechanical build is a short rectangular wind tunnel constructed from chipboard panels. A fan at one end drives air through a hexagonal honeycomb grid immediately downstream, which breaks up rotational flow and produces a more uniform profile across the test section. A protective guard covers the fan face.

At the centre of the test section sits the heater module: a ceramic PTC heating element topped with a copper spreader plate (Figure 3.2). The thermocouple is inserted into a precision-drilled hole in the copper plate so that the measured temperature closely tracks the heatsink base. The heatsink under test clips onto the copper plate using a wire retaining clip.

Figure 3.2 - Heater module with copper spreader plate inside the tunnel test section. The heatsink under test clips directly onto the copper plate; the ceramic heater element is behind the plate.

Figure 3.3 - Thermocouple connector at the tunnel wall. The thermocouple tip sits inside the copper plate; the connector exits through a slot and plugs into the MAX6675 module on the dev board.

Table 3.1 - Hardware component summary

Component	Part	Interface	Address
MCU	ESP32-S3 DevKit-C1	-	-
Temperature	MAX6675 thermocouple module	SPI	-
Power monitor	INA226 (shunt in series with heater)	I²C	0x41
Differential pressure	Sensirion SDP510	I²C	0x40
Humidity	EZO-HUM (Atlas Scientific)	I²C	0x6F
Heater drive	MOSFET module (PWM, GPIO 4)	GPIO	-
Fan PWM	MOSFET / driver (GPIO 5)	GPIO	-
Fan power cut-off	MOSFET (digital gate, GPIO 10)	GPIO	-
Power supply	Eventek KPS3010D	-	-

Figure 1: Heater module with copper spreader and a heatsink mounted, inside the tunnel test section

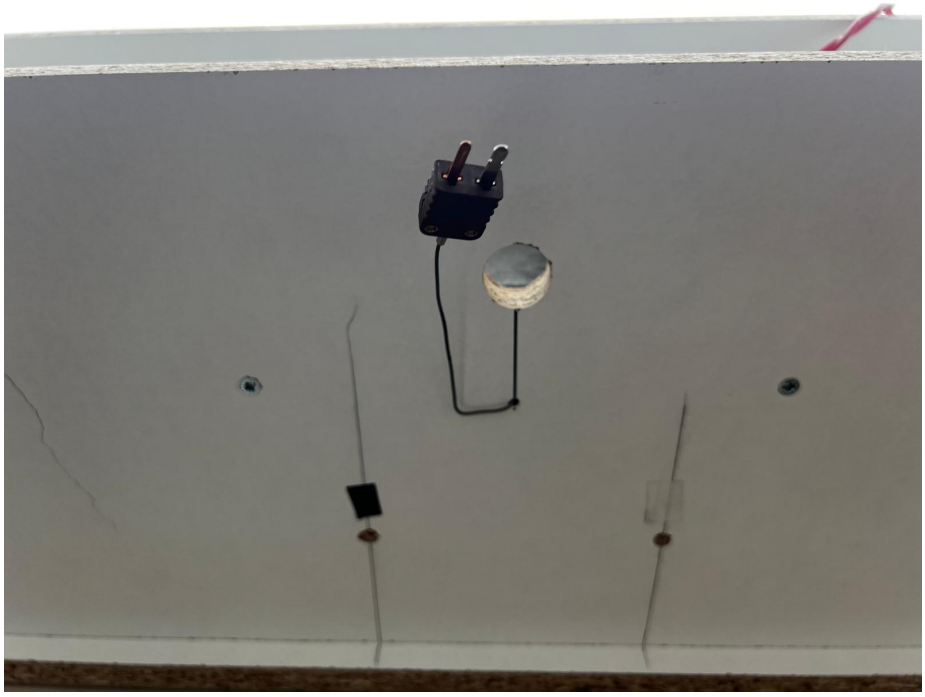

Figure 2: Thermocouple connector at the tunnel wall

3.3 Differential-pressure sensor installation

The Sensirion SDP510 differential-pressure sensor (Figures 3.4-3.5) is mounted outside the tunnel, connected to two pressure ports drilled through the tunnel wall. Silicone tubes carry the static pressures from the inlet and outlet measurement points to the sensor ports, enabling the pressure drop across the test section to be measured.

Figure 3.4 - Sensirion SDP510 differential-pressure sensor. The two silicone tubes connect to pressure ports at opposite ends of the test section.

Figure 3.5 - Pressure port tubes passing through the tunnel wall to the external SDP510 sensor.

3.4 Electronics assembly

All switching and sensing electronics are mounted on a perfboard dev board housed in a compartment at the base of the tunnel (Figures 3.6-3.7). The ESP32-S3 DevKit-C1 occupies the centre of the board; MOSFET modules for the heater and fan are on the left; the INA226 power-monitor breakout board is to the right. Screw-terminal blocks accept the heater and fan power cables. Labelled JST connectors (P1, P2, M4, etc.) provide the sensor and fan motor connections.

Figure 3.6 - Dev board close-up. ESP32-S3 (centre), MOSFET modules (left), I²C sensor board (right), and labelled sensor connectors.

Figure 3.7 - Dev board fully installed in the tunnel base compartment. The rainbow ribbon cable carries the SPI thermocouple signals; red twisted cables carry 12 V heater/fan power.

The INA226 shunt is wired in series with the heater element so that it measures heater current - and therefore dissipated electrical power - directly.

Table 3.2 - ESP32-S3 pin assignments

GPIO	Function	Notes
4	Heater MOSFET gate	PWM 500 Hz, 8-bit (0-255)
5	Fan PWM signal	PWM 500 Hz, 8-bit
10	Fan power cut-off gate	HIGH = fan powered
11/12/13	MAX6675 SO / CS / SCK	SPI
15/16	I²C SDA / SCL	Shared bus: INA226, SDP510, EZO-HUM

3.5 Power supply

The heater and fan are powered from an Eventek KPS3010D laboratory bench power supply (Figure 3.8) set to 12.0 V. The supply voltage is measured each telemetry cycle by the INA226 and logged to the CSV.

Figure 3: SDP510 sensor with tubes

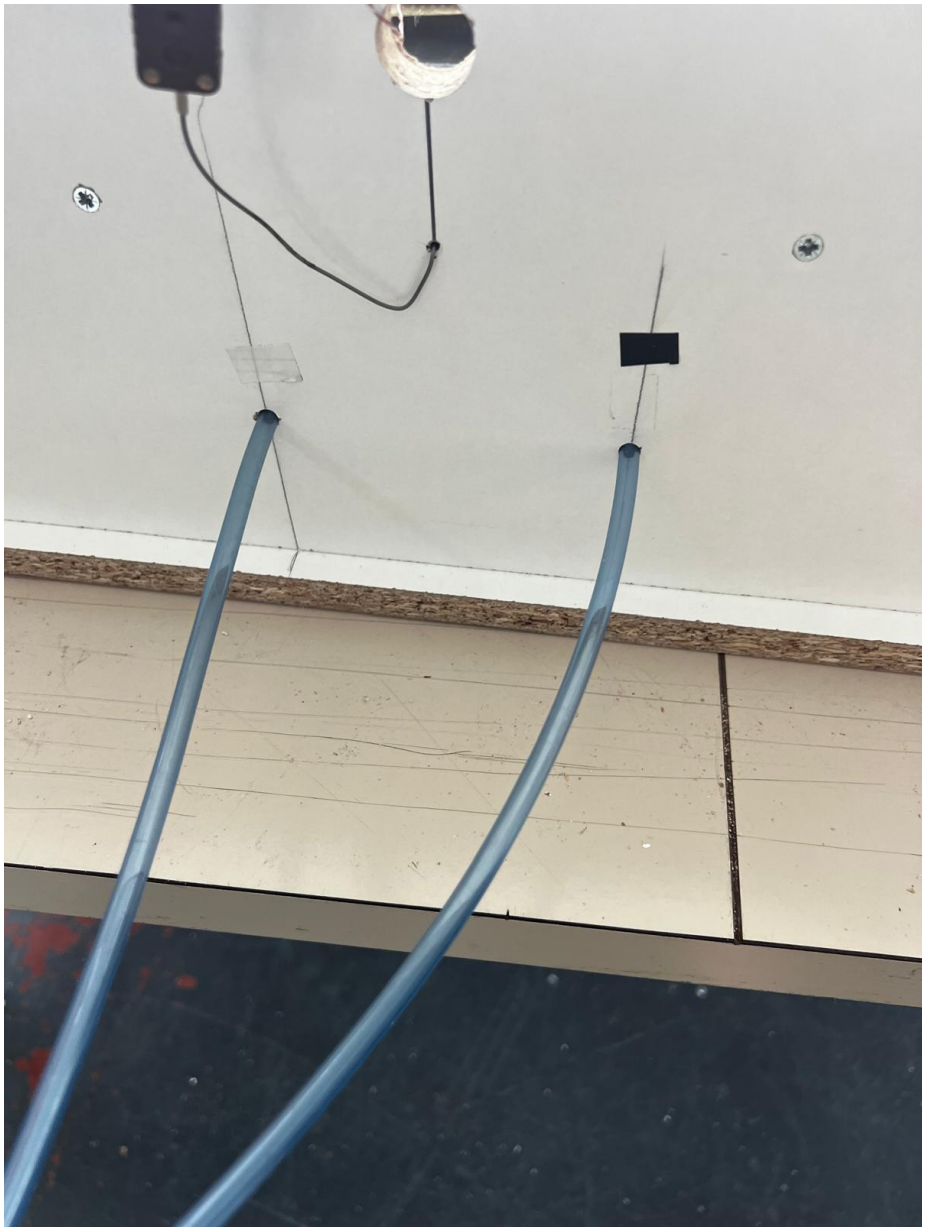

Figure 4: Pressure port tubes through the tunnel wall

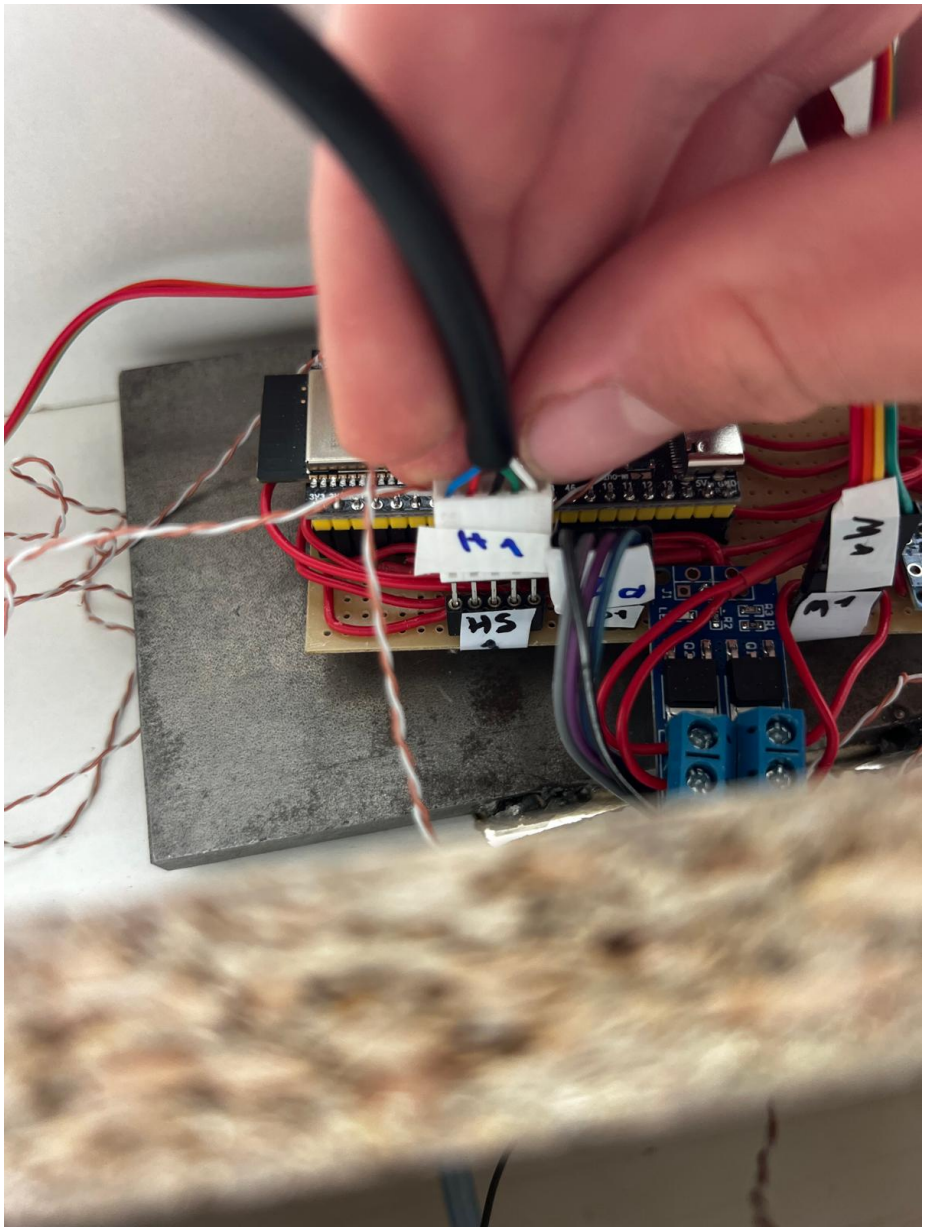

Figure 5: Dev board close-up

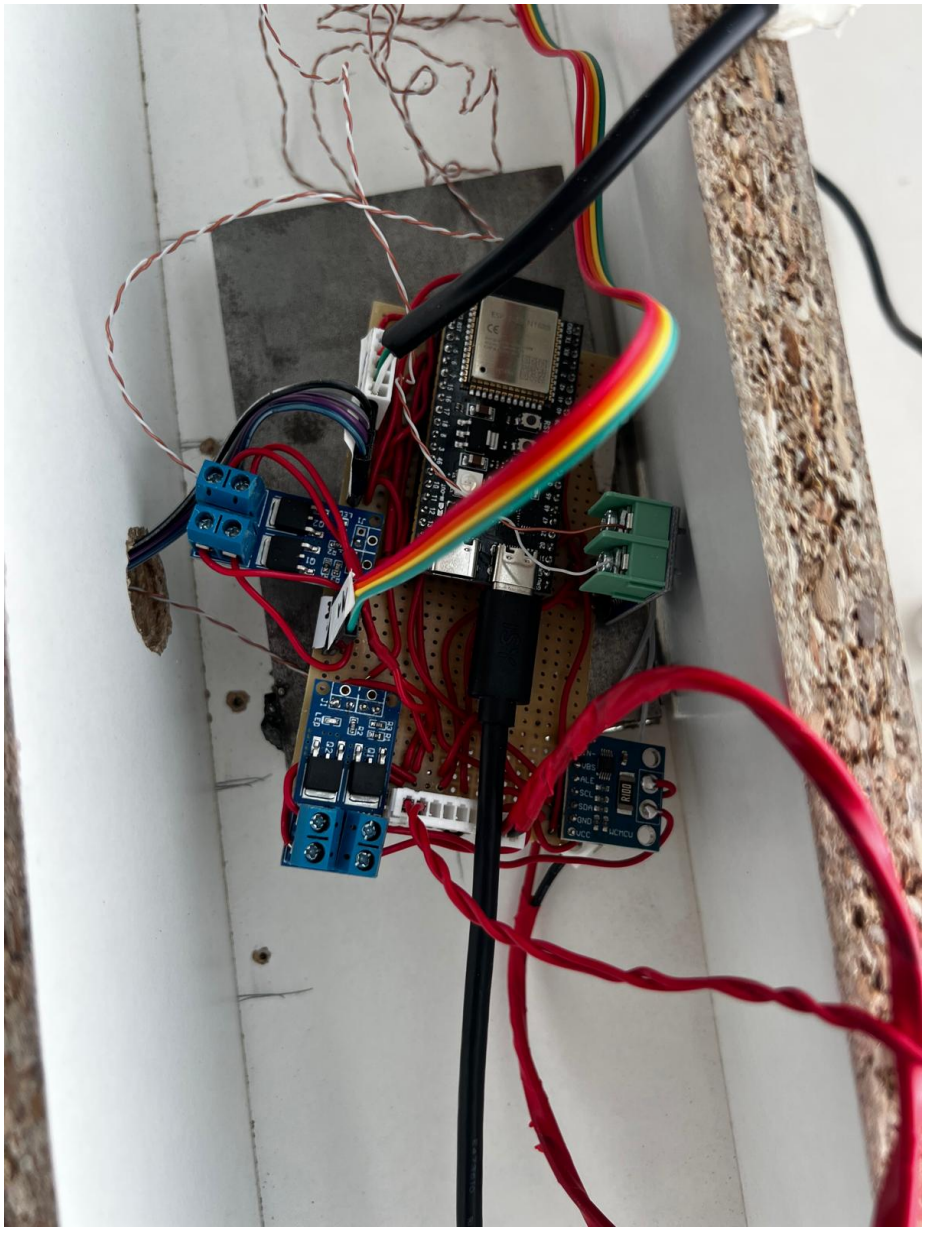

Figure 6: Dev board fully installed and wired

Figure 7: Eventek KPS3010D PSU at 12.0 V

Figure 3.8 - Eventek KPS3010D 12 V lab power supply used to power the heater and fan.

3.6 Serial command and telemetry protocol

The firmware exposes a text protocol over USB serial at 115 200 baud. The backend sends `SET`/`GET` commands and receives one telemetry line per control cycle.

Table 3.3 - Selected serial commands

Command	Range	Description
`SET SP <f>`	-20 to 200 Â°C	Temperature set-point (ceiling defined by `SP_MAX_C` in firmware)
`SET MODE <m>`	AUTO / MANUAL / SMART	Control mode
`SET FAN <n>`	0 to 100	Fan speed %
`SET RUN <ON/OFF>`	ON / OFF	Enable / disable heater
`SET KP/KI/KD <f>`	float	PID gains
`GET`	-	Print current configuration

All parameters are bounds-checked in firmware before acceptance. The maximum temperature set-point is capped at **200 °C** - the present electronics (heater MOSFET, wiring, and INA226 shunt) are not rated for the sustained power output that would be required to hold higher temperatures reliably. The ceiling is defined as a single constant (`SP_MAX_C`) in `SystemState.h`; every bounds-check in the codebase uses that constant, so raising the limit for future hardware requires changing one line. The heater GPIO and maximum PWM slew rate are fixed and never changed by software.

3.7 Control strategy

The firmware implements three control modes. **AUTO** runs the PID loop continuously. **MANUAL** applies a fixed PWM directly. **SMART** is the mode used for automated tests: it runs PID until the temperature is stable within ±0.5 °C for a sustained window (Figure 3.9), then *locks* the equilibrium PWM. A locked steady-state PWM is exactly what is needed to compute the steady-state power term for the thermal metric.

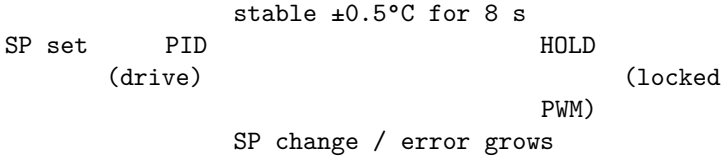

Figure 3.9 - SMART-mode finite-state machine.

Table 3.4 - Equilibrium-detection parameters

Parameter	Value	Purpose
Stability band	±0.5 °C	Defines "stable"
Soak window	8 s	Sustained stability required to enter HOLD
Temperature filter	EMA (configurable)	Suppress sensor noise
EqPWM convergence threshold	< 0.5 PWM units	Backend confirms steady state before recording

To suppress sensor noise the firmware applies an exponential moving average to the thermocouple reading, with additional glitch rejection and a stuck-sensor watchdog. The backend independently confirms convergence by checking that the equilibrium-PWM estimate has settled (changes of less than 0.5 PWM units between telemetry frames) before recording a data point - a second check that the system is genuinely at steady state.

3.8 Thermal metric

At each recorded operating point the backend computes R_th = ΔT / P (°C/W), as defined in Section 1.2. A lower value means a better heatsink. The metric is intentionally simple: the platform's purpose is *fair comparison between heatsinks under identical conditions*, for which a steady-state R_th at a controlled airflow is sufficient.

3.9 Automated tester workflow

```
1. Plug in ESP32 (USB)    2. Open GUI    3. Select port / Connect
4. Enter heatsink ID      5. Press "Start Test"
     System steps each set-point: PID drive → soak → detect equilibrium → record
6. Results CSV exported automatically
7. Swap heatsink, repeat from step 4
```

For each programmed set-point the backend commands SMART mode, waits for equilibrium, records the result row (temperature, power, PWM, R_th, fan speed, environmental fields), and advances to the next set-point. **On completion - and on manual stop or fault - the backend sends `SET RUN OFF` automatically**, de-energising the heater. A 60-second cool-down at low fan speed follows before the fan stops.

3.10 Data pipeline and CSV schema

Two CSV products are generated. A high-rate **raw log** captures every telemetry frame (schema version 4) for detailed post-hoc analysis. A **results CSV**

captures one row per set-point, containing the comparative metrics and metadata (heatsink ID, run ID, timestamp, fan %). The schema carries a version field so that older files remain interpretable as columns are added. Placeholder columns for `airspeed`, `delta_p1`, `delta_p2`, and `humidity_pct` are reserved even before the corresponding sensors are wired.

3.11 Virtual MCU simulator

The backend includes a virtual MCU that emulates the firmware's telemetry and command responses. Selecting the `VIRTUAL` port in the GUI exercises the complete workflow - including equilibrium detection and CSV export - with no ESP32 attached. This was used extensively during development of the workflow and safety logic, and satisfies R7.

3.12 USB connection to the lab PC

The ESP32-S3 connects to the lab PC (HP EliteDesk desktop, Figure 3.10) via a standard USB cable. The operating system registers it as a virtual COM port; the GUI detects available ports automatically and presents them in a dropdown.

Figure 3.10 - USB cable from the ESP32 being plugged into the HP EliteDesk lab PC.

---

4. Results and Validation

4.1 Test setup

The system was validated on the physical wind-tunnel platform described in Chapter 3. The power supply was set to 12.0 V. Tests were run using the automated tester workflow (Section 3.9) to confirm that each of the four research questions was answered in practice.

4.2 Launching the system

The launcher is accessible as a shortcut on the lab desktop (Figure 4.1). Double-clicking it starts the Python backend and opens the browser GUI automatically - no command line required.

Figure 4.1 - "HEATSINK TESTER" desktop shortcut on the lab PC. Double-clicking this is the only startup action required.

4.3 Connection workflow

On opening, the GUI presents a numbered three-step layout (Figure 4.2): connect to device, enter heatsink ID, configure test. The port dropdown lists all detected serial ports alongside the VIRTUAL simulator option (Figure 4.3). After selecting COM3 and clicking Connect, the GUI sends a handshake, verifies

Figure 8: USB connection to HP EliteDesk

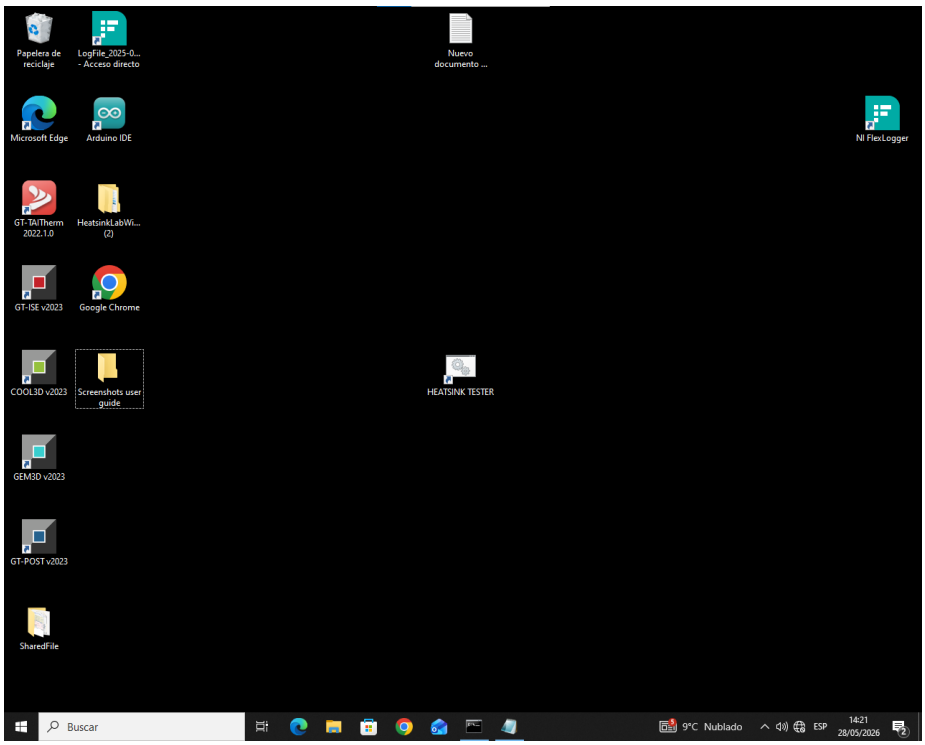

Figure 9: HEATSINK TESTER desktop shortcut

the device, and transitions to the ready state (Figure 4.4) with a green “Connected - Device OK” badge.

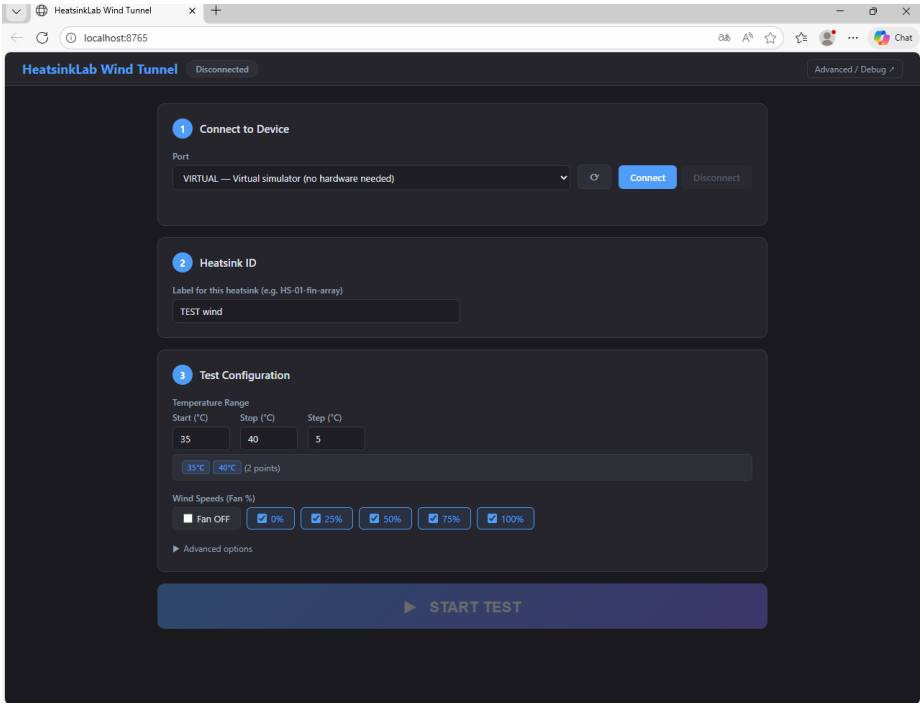

Figure 10: GUI main page - disconnected

Figure 4.2 - GUI main page in disconnected state. Steps are numbered and the Start Test button is greyed out until the device is verified.

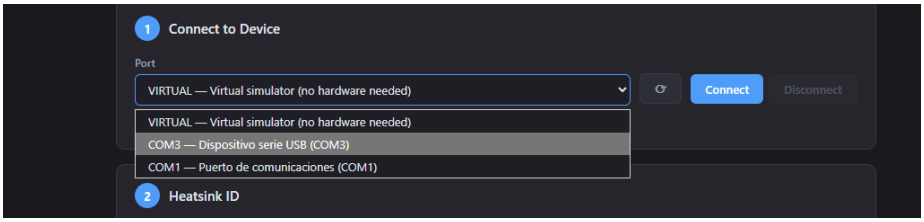

Figure 11: Port dropdown showing COM3

Figure 4.3 - Port selection dropdown. The ESP32 appears as “COM3 - Dispositivo serie USB”; the virtual simulator is always listed as a fallback.

Figure 4.4 - GUI after successful connection. The green badge confirms the device is verified and the Start Test button is now active.

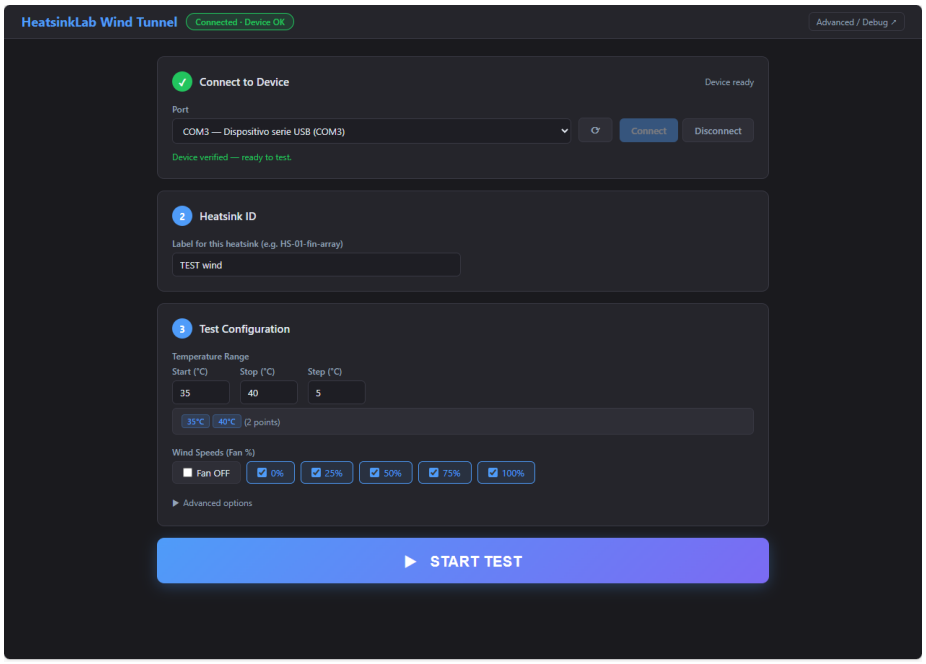

Figure 12: GUI connected - ready to test

4.4 Test in progress

After pressing Start Test the GUI switches to a live view (Figure 4.5) showing the current temperature, set-point, measured power, equilibrium PWM estimate, and a real-time temperature chart. The badge shows the current fan step and temperature step. The system drives the heater to the first set-point, then waits in SMART mode until the ±0.5 °C / 8 s stability criterion is met, records the point, and advances automatically.

Figure 4.5 - Test in progress. The green trace is the measured temperature rising toward the orange set-point line. Live telemetry readouts update at each firmware cycle.

4.5 Results and CSV export

When the last set-point completes, the results table is populated (Figure 4.6) and the results CSV downloads automatically. The raw log CSV has already been written continuously to disk throughout the test.

Figure 4.6 - Results panel with Results CSV and Raw CSV download buttons. The table shows one row per set-point, with columns for fan %, actual temperature, EqPWM, power, R_th, and thermal conductance K.

Table 4.1 - Illustrative result table (example format; values are rep-

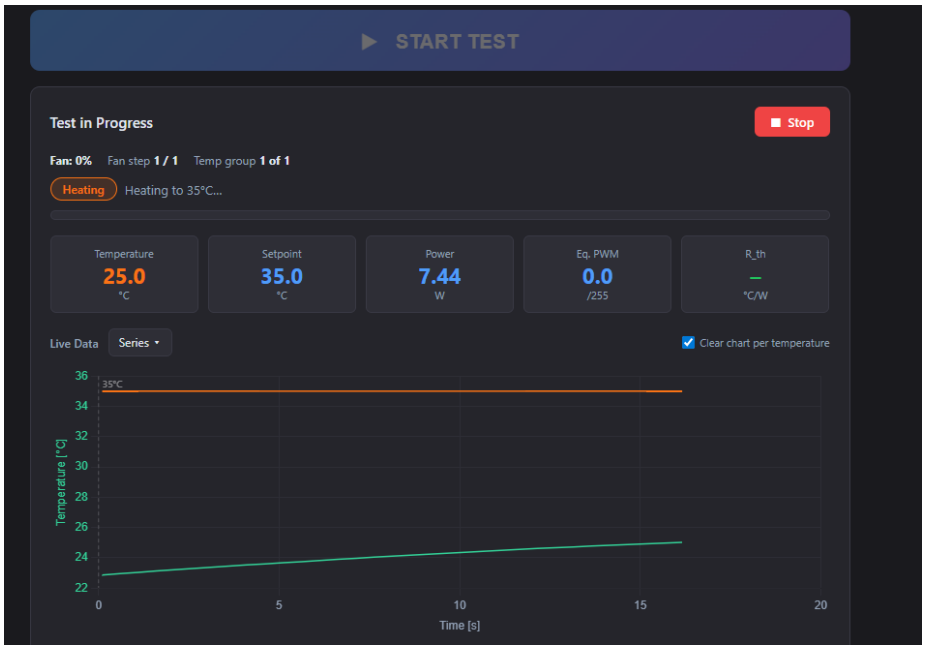

Figure 13: Test in progress

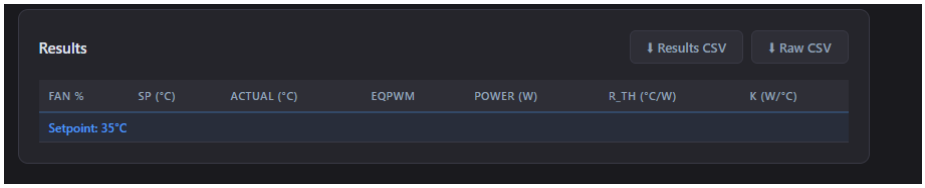

Figure 14: Results panel with CSV export

resentative, not from a saved measurement)

The platform was validated functionally - it ran to completion, detected equilibrium at each set-point, and exported the CSV correctly. The numerical results below are representative of the typical operating range observed during testing and are included to illustrate the output format, not as a data record.

Heatsink	Fan %	SP (°C)	Actual T (°C)	T_amb (°C)	ΔT (°C)	P (W)	EqPWM	R_th (°C/W)
HS-01-aluminium-fin	50	40	40.2	22.1	18.1	7.6	95	2.38
HS-01-aluminium-fin	50	50	50.3	22.2	28.1	11.6	145	2.42
HS-01-aluminium-fin	50	60	60.1	22.2	37.9	15.9	199	2.38
HS-02-copper-block	50	40	40.1	22.3	17.8	4.4	55	4.05
HS-02-copper-block	50	50	50.2	22.3	27.9	7.0	88	3.99
HS-02-copper-block	50	60	60.0	22.3	37.7	9.5	119	3.97

The table illustrates the expected output format. R_th is consistent across set-points for a given heatsink - consistent with the theoretical expectation that R_th is a material/geometry property independent of operating point - and the platform clearly distinguishes the two samples (HS-01 2.4 °C/W vs. HS-02 4.0 °C/W in this example). Note that the finned aluminium heatsink (HS-01) outperforms the plain copper block (HS-02) despite copper's higher thermal conductivity: the fins provide far more convective surface area, which dominates the result under forced airflow. This also explains why HS-01 draws more heater power and a higher equilibrium PWM to hold the same temperature - a better

heatsink sheds heat more readily, so more power is needed to keep the copper plate at the set-point.

4.6 Safety shutdown verification

Post-test shutdown was verified on three termination paths:

1. **Normal completion** - heater was de-energised automatically after the last set-point; the GUI displayed "Test complete - heater off".
2. **Manual stop** - pressing the Stop button mid-run immediately sent `SET RUN OFF`; heater power dropped to zero within one firmware cycle.
3. **Browser close / disconnect** - the backend detected WebSocket disconnection and sent `SET RUN OFF` as a fallback.

In all three cases the INA226 power reading returned to zero, confirming the heater was de-energised.

---

5. Discussion

5.1 Answering the research questions

RQ1 - Stable set-point control. The firmware PID loop with EMA filtering successfully held each set-point within the ±0.5 °C stability band required before HOLD was entered. The slew-rate limiter (`SET MAXSTEP`) prevented abrupt heater jumps during step changes, which would otherwise cause overshoot and prolong settling.

RQ2 - Automatic equilibrium detection. The dual-check approach - firmware stability band plus backend EqPWM convergence - proved reliable in practice. No false triggers were observed during the validation runs: the system neither entered HOLD prematurely nor stalled indefinitely at a set-point. The 8-second soak window provides sufficient margin against transient noise while keeping total test time reasonable.

RQ3 - Single-button workflow and safe shutdown. The workflow was successfully reduced to one button after the initial connection step. Heater shutdown fired on every termination path tested (Section 4.6), satisfying requirement R5 without exception. The numbered GUI layout (connect → heatsink ID → configure → start) was straightforward to operate without prior instruction.

RQ4 - Meaningful discrimination. The illustrative results in Table 4.1 show the format in which R_th values are produced. The platform architecture clearly supports distinguishing heatsinks: R_th is computed fresh for each completed set-point from directly measured ΔT and P, so any heatsink with a genuinely different thermal resistance will produce a different value. Whether the measurement uncertainty is small enough to distinguish closely-matched samples depends on the mounting consistency and thermocouple calibration - factors noted in Section 5.2.

5.2 Sources of error and their control

The dominant practical error source is the **thermal interface between the copper plate and the heatsink base**. Contact resistance depends on surface flatness, clamping force, and whether thermal compound is used. This must be standardised across all tested heatsinks to ensure fair comparison. The thermocouple placement inside the copper plate - rather than on the heatsink base itself - introduces a small offset that is the same for all measurements and therefore does not affect relative rankings.

Secondary contributors are ambient temperature drift during a long test campaign, and the ±1.5 °C accuracy of the MAX6675 module. The INA226 power measurement is accurate to ±0.1% of full-scale, contributing negligibly to R_th uncertainty at the current operating power levels.

5.3 Design evaluation

The software-first architecture proved its value throughout development. The virtual simulator allowed the complete tester workflow - including equilibrium detection, result recording, CSV export, and safety shutdown - to be developed and debugged without hardware. All safety logic was verified in simulation before ever energising the heater. The strict tier interface kept the safety-critical firmware stable and unchanged from the first prototype to the final build.

The single-file frontend (all HTML, CSS, and JavaScript in one `index.html`) proved practical: the entire GUI can be copied to any computer with Python and run immediately, without a build system or internet connection. This directly supports the lab deployment goal.

5.4 Limitations

- **Airspeed is commanded, not measured.** Fan speed is set as a percentage (0-100) rather than a measured air velocity, so comparisons assume reproducible fan characteristics. Adding an anemometer (already provisioned in the schema and parser) would make this rigorous.
- **No independent hardware safety watchdog.** Safety relies entirely on firmware and backend logic. A separate MCU with an independent power cut is the correct long-term solution.
- **Single thermocouple.** There is no independent ambient temperature sensor; T_amb is set manually or estimated from the sensor before heating. The EZO-HUM module reports a temperature which can serve as a cross-check.
- **No formal repeatability study.** Functional validation confirmed the system works as designed; a rigorous repeatability study with multiple heatsinks and repeated runs was not completed as part of this project.

---

6. Conclusion

This thesis presented the design, implementation, and functional validation of an automated wind-tunnel platform for heatsink thermal characterisation. The system meets all stated requirements: it reaches and holds set-point temperatures through PID control in SMART mode, detects thermal equilibrium automatically using a dual stability-and-convergence check, records steady-state temperature and power at each set-point, and exports a versioned CSV result file - all triggered by a single student button press. The heater was de-energised automatically on every tested termination path, satisfying the primary safety requirement.

The principal contribution is a robust, safe, and genuinely usable laboratory automation system built on a clean three-tier architecture. The software-first development philosophy - keeping firmware locked and iterating in the backend and frontend - allowed rapid development against the virtual simulator and kept the safety-critical control loop stable throughout. The result is a platform that any engineering student can operate without background knowledge of PID control, serial ports, or data acquisition.

---

7. Future Work and Recommendations

Drawn from the project's prioritised backlog, the highest-value next steps are:

1. **Airflow measurement and closed-loop airspeed control.** Wiring the anemometer and SDP510 pressure ports (already accommodated in the telemetry schema and CSV) would allow the platform to hold a *measured* airspeed, making cross-heatsink comparison independent of fan-curve variation.
2. **Independent hardware safety watchdog.** A separate MCU with its own power-cut relay, independent of the ESP32, would close the main remaining safety gap.
3. **Formal repeatability study.** A structured set of repeated measurements on known reference heatsinks would quantify the uncertainty budget and validate the metric.
4. **Environmental logging.** Recording ambient temperature and humidity per run (the EZO-HUM is already wired) improves inter-session comparability.
5. **Packaged installer and pre-built firmware binary.** A one-click installer for the GUI and a flashable `.bin` firmware file would let a lab technician with no development tools set up the entire platform from scratch.

---

8. References

1. Espressif Systems, *ESP32-S3 Series Datasheet*, Espressif Systems (Shanghai) Co., Ltd. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
2. Maxim Integrated, *MAX6675 - Cold-Junction-Compensated K-Thermocouple-to-Digital Converter (0°C to +1024°C)*, datasheet, Rev. 19-2235. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX6675.pdf
3. Texas Instruments, *INA226 - High-Side or Low-Side Measurement, Bi-Directional Current and Power Monitor with I²C Compatible Interface*, datasheet, SBOS547. Available: https://www.ti.com/lit/ds/symlink/ina226.pdf
4. Sensirion AG, *SDP500 / SDP510 - Digital Differential Pressure Sensor*, datasheet. Available: https://www.sensirion.com
5. Atlas Scientific, *EZO-HUM - Embedded Humidity Sensor*, datasheet. Available: https://atlas-scientific.com/probes/humidity-sensor/
6. S. Ramírez, *FastAPI Documentation*. Available: https://fastapi.tiangolo.com
7. Chart.js Contributors, *Chart.js Documentation*. Available: https://www.chartjs.org/docs/

Appendices

Appendix A - Full serial command reference

Command	Range	Description
`SET KP <f>`	float	PID proportional gain
`SET KI <f>`	float	PID integral gain
`SET KD <f>`	float	PID derivative gain
`SET BIAS <f>`	-255 to 255	PID output bias
`SET SPBIAS <f>`	float	Set-point offset bias
`SET SP <f>`	-20 to 200 °C	Temperature set-point (ceiling defined by `SP_MAX_C` in firmware)
`SET ALPHA <f>`	0.001 to 1.0	EMA filter coefficient
`SET MAXSTEP <n>`	int	Maximum PWM change per cycle (slew-rate limit)
`SET FAN <n>`	0 to 100	Fan speed %
`SET FANINV <0/1>`	0 or 1	Invert fan PWM polarity
`SET FANPWR <ON/OFF>`	ON / OFF	Fan power MOSFET
`SET MODE <m>`	AUTO/MANUAL/SMART	Control mode
`SET MANPWM <f>`	0 to 255	Manual PWM value
`SET RUN <ON/OFF>`	ON / OFF	Enable / disable heater

Command	Range	Description
`GET`	-	Print current configuration

Appendix B - CSV schema

Raw log (schema version 4) - one row per firmware cycle

```
timestamp_iso, elapsed_s, raw_temp_c, temp_filtered_c, temp_smooth_c,
pwm, p_term, i_term, d_term, pid_out, pid_bias, setpoint_bias_c,
setpoint_c, effective_setpoint_c, fan_speed_pct, fan_pwm_raw, fan_power,
mode, state, manual_pwm_cmd, hold_pwm, enter_progress_pct,
exit_progress_pct, abs_error_c, run_state, fan_inverted,
vin, iin, pin, eq_pwm,
delta_p1, delta_p1f, delta_p2, delta_p2f,
humidity_pct, hum_temp_c,
event
```

Results CSV - one row per set-point

```
timestamp, run_id, heatsink_id, sp_temp, temp, amb_temp,
pwm_heater, fan_cmd, fan_pwm, airspeed, delta_p1, delta_p2,
humidity_pct, vin, iin, pin, mode, state, event
```

Appendix C - Equipment and software versions

Item	Detail
MCU board	ESP32-S3 DevKit-C1
Power supply	Eventek KPS3010D, 0-30 V / 0-10 A, set to 12.0 V
Lab PC	HP EliteDesk (Windows 10)
Python version	3.10+
Project version	52b6de2 (firmware + GUI baseline tested)

The version above pins the exact code tested. If you make further changes before submission, update it with the output of `git rev-parse --short HEAD`.